TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM

HCMUTE

MACHINE VISION PROJECT REPORT

TOPIC: Badminton Player Activity Heatmap

GROUP MEMBER NAMES AND ID

PHẠM VĂN THỊNH - 22146056

TRẦN ANH KHƯƠNG - 22146025

MACHINE VISION PROJECT REPORT

Topic: Badminton Player Activity Heatmap

I. Introductions:

1. Background:

In competitive badminton, understanding player movement patterns is crucial for tactical analysis, performance optimization, and opponent scouting. Coaches and analysts need to identify which areas of the court a player dominates, their movement efficiency, and positional tendencies during rallies.

2. Problem statement:

Video footage from standard cameras captures matches from an oblique (side or elevated) angle, creating perspective distortion. This distortion means that:

-Players near the camera appear larger and occupy more pixels.

-Players far from the camera appear smaller.

-Direct pixel coordinates cannot accurately represent real court positions.

Additionally, No advanced ML models are used like Yolo or Mediapipe.

3. Objectives:

-**Manually calibrate** the court using homography (by clicking 4 corners).

-**Detect players** using traditional computer vision (background subtraction/motion detection).

-**Extract foot positions** (lowest point of player blob).

-**Transform pixel coordinates** to real-world court space.

-**Generate heatmap** over 10 minutes of match segments.

4. Scope and Limitations:

Input: Static camera video (10 minutes, 30 fps, caulong1.mp4).

Output: JET-colormap heatmap overlaid on a standard badminton court diagram.

Assumptions:

-Camera is fixed (no pan/tilt/zoom during recording).

-Court corners are visible in the first frame.

-Players are the largest moving objects on court.

II. Literature Review & Theoretical Background

1. Perpective Transformation (Homography):

In computer vision, Homography is a projective transformation that maps points from one plane to another. For this project, we map the camera image plane to the real-world court plane.

Mathematical Formulation:

The transformation is defined by a 3×3 matrix H:

$$\begin{bmatrix} X' \\ Y' \\ W' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Where:

(x, y): pixel coordinates of camera image

(X, Y): real world coordinates

2. Motion detection via background subtraction

Since deep learning models are not allowed, we use Background Subtraction to identify moving objects:

-Algorithm: MOG2 (Mixture of Gaussians) from OpenCV (cv.createBackgroundSubtractorMOG2).

-Principle: Each pixel is modeled as a mixture of Gaussian distributions. Pixels that deviate significantly from the background model are classified as "foreground" (moving objects).

Parameters:

-**history = 500:** Number of frames used to build the background model.

-**varThreshold = 50:** Sensitivity threshold for foreground detection.

3. Contour detection and player filtering:

After obtaining the foreground mask:

-Morphological operations remove noise (opening, closing, dilation).

-Contour detection (cv.findContours) identifies connected regions.

Filtering by:

-Area (>800 pixels): Removes small objects (shuttlecock, noise).

-Aspect ratio (height>width): Filters out non-human shapes.

-Minimum height (30 pixels): Removes small false positives.

4. Foot position estimations

The foot position is crucial because it represents the player's actual contact point with the court surface. Given a bounding box (x,y,w,h):

-Foot X = x+w/2 (center of the bounding box width).

-Foot Y = y+h (bottom of the bounding box).

5. Heatmap generation

A heatmap visualizes spatial density:

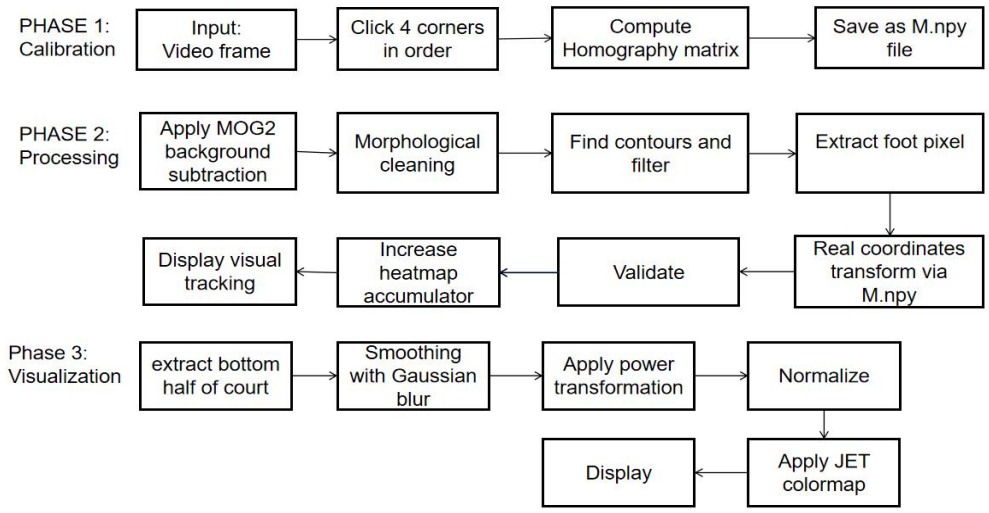
-**Accumulation**: Each transformed foot position increments a 2D accumulator array.

-**Smoothing**: Gaussian blur ($\sigma = 20$, kernel size 81×81) creates continuous density distribution.

-**Normalization**: Values are scaled to [0, 255] for visualization.

-**Colormap**: OpenCV's COLORMAP_JET maps density values to colors (blue = low, red = high).

III. System workflow



IV. Detail Implementations:

1. Calibration Module (in setup_perspective.py file)

Purpose: Compute the transformation matrix MM that maps camera pixels to real-world court coordinates.

Process:

-Load the first frame of the video.

-Display the frame at a manageable resolution (960px width).

-User clicks the 4 corners of the court in exact order:

+Corner 1: Top-Left (far left, near net)

+Corner 2: Top-Right (far right, near net)

+Corner 3: Bottom-Right (near camera, right side)

+Corner 4: Bottom-Left (near camera, left side)

Each click draws a marker and label on the image.

After collecting 4 points, compute:

```
src = np.float32(points)
    dst = np.float32([
        [0,        0       ],
        [COURT_W,  0       ],
        [COURT_W,  COURT_H ],
        [0,        COURT_H ]
    ])
    M = cv.getPerspectiveTransform(src, dst)
    np.save(OUTPUT_M, M)
```

2. Player detection module:

```
def detect_players(fg_mask, min_area=MIN_AREA):

    # Lọc nhiễu morphology
    kernel = cv.getStructuringElement(cv.MORPH_ELLIPSE, (7, 7))
    mask = cv.morphologyEx(fg_mask, cv.MORPH_OPEN,  kernel)
    mask = cv.morphologyEx(mask,    cv.MORPH_CLOSE, kernel)
    mask = cv.dilate(mask, kernel, iterations=2)

    contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL,
                                  cv.CHAIN_APPROX_SIMPLE)
    boxes = []
    for cnt in contours:
        area = cv.contourArea(cnt)
        if area < min_area:
            continue
        x, y, w, h = cv.boundingRect(cnt)
        if h < w:           # người đứng thì h > w
            continue
        if h < 30:          # quá thấp → không phải người
            continue
        boxes.append((x, y, w, h))
    return boxes
```

3. Coordinates transformation module:

```
def transform_point(M, fx, fy):
    pt = np.float32([[[fx, fy]]])
    out = cv.perspectiveTransform(pt, M)
    rx = int(out[0][0][0])
    ry = int(out[0][0][1])
   return rx, ry
```

4.Heatmap Accumulation

```
heatmap_acc = np.zeros((COURT_H, COURT_W), dtype=np.float32)

    frame_count    = 0
    positions_log  = []
    print('[START] Bat dau xu ly video... Nhan Q de dung som.')
    while True:
        ret, frame = cap.read()
        if not ret:
            print('[INFO] Het video hoac khong doc duoc frame.')
            break
        if frame_count >= max_frames:
            print(f'[INFO] Du {DURATION_MIN} phut. Dung xu ly.')
            break
        fg_mask = bg_sub.apply(frame)
        boxes   = detect_players(fg_mask)
        display = frame.copy()
        frame_h = frame.shape[0]
        split_y = int(frame_h * BOTTOM_HALF_RATIO)
        for bbox in boxes:
            x, y, w, h = bbox
            center_y = y + h // 2
            if center_y < split_y:
                continue
            frame_w  = frame.shape[1]
            center_x = x + w // 2
            if center_x < int(frame_w * CENTER_X_MIN) or center_x > int(frame_w *
CENTER_X_MAX):
                continue
            cv.rectangle(display, (x,y), (x+w, y+h), COLOR_BOX, 2)
            fx, fy = get_foot_point((x, y, w, h))
            cv.circle(display, (fx, fy), 5, COLOR_FOOT, -1)
            rx, ry = transform_point(M, fx, fy)
            if 0 <= rx < COURT_W and 0 <= ry < COURT_H:
                heatmap_acc[ry, rx] += 1
```

5. Visualization

```
HALF_Y = COURT_H // 2
acc_half = acc[HALF_Y:, :]  #bottom half of frame extract
blur = cv.GaussianBlur(acc_half, (81, 81), 20) #larger kernal like 81x81 ensure
smooth
blur = np.power(blur, 0.4) #power transformation
norm = cv.normalize(blur, None, 0, 255, cv.NORM_MINMAX)
norm_uint8 = np.uint8(norm)
colored = cv.applyColorMap(norm_uint8, cv.COLORMAP_JET)
```

V. Implementation details

1. Key parameters

Parameter	Value	Description
COURT_W	610	In pixels (milimeters)
COURT_H	1340	In pixels
DURATION_MIN	10	Minutes
MIN_AREA	800	Minimum player contour area
BOTTOM_HALF_RATIO	0.5	Only 50% of bottom frame is processing
CENTER_X_MIN	0.2	Ignore left 20% frame
CENTER_X_MAX	0.8	Ignore right 20% frame

2. Perform consideration:

Processing Speed:

-MOG2 background subtraction: ~5-10 ms per frame.

-Contour detection: ~2-5 ms per frame.

-Heatmap update: negligible.

-Total: ~10-15 ms/frame → ~60-100 FPS processing capability.

Memory Usage:

-Heatmap accumulator: 1340 × 610 × 4 bytes = ~3.3 MB.

-Frame buffers: ~2-3 MB per frame.

VI. Code Walkthrough (main.py program)

1. Main processing loop

```
def main():
    try:
        M = np.load(M_PATH)
        print(f'[OK] Da load ma tran M tu {M_PATH}')
    except FileNotFoundError:
        print(f'[LOI] Khong tim thay {M_PATH}')
        print('     Hay chay perspective_setup.py truoc!')
        return
    cap = cv.VideoCapture(VIDEO_PATH)
    if not cap.isOpened():
        print(f'[LOI] Khong mo duoc video: {VIDEO_PATH}')
        return
    fps = cap.get(cv.CAP_PROP_FPS)
    if fps <= 0:
        fps = 30
    max_frames = int(fps * 60 * DURATION_MIN)
    print(f'[INFO] FPS={fps:.1f} | Se xu ly {max_frames} frames ({DURATION_MIN}
phut)')
    bg_sub      = cv.createBackgroundSubtractorMOG2(
                      history=500, varThreshold=50, detectShadows=True)
    heatmap_acc = np.zeros((COURT_H, COURT_W), dtype=np.float32)
    frame_count    = 0
    positions_log  = []
    print('[START] Bat dau xu ly video... Nhan Q de dung som.')
    while True:
        ret, frame = cap.read()
        if not ret:
            print('[INFO] Het video hoac khong doc duoc frame.')
            break
        if frame_count >= max_frames:
            print(f'[INFO] Du {DURATION_MIN} phut. Dung xu ly.')
            break
        fg_mask = bg_sub.apply(frame)
        boxes   = detect_players(fg_mask)
        display = frame.copy()
        frame_h = frame.shape[0]
        split_y = int(frame_h * BOTTOM_HALF_RATIO)
        for bbox in boxes:
            x, y, w, h = bbox
            center_y = y + h // 2
            if center_y < split_y:
                continue
            frame_w  = frame.shape[1]
            center_x = x + w // 2
            if center_x < int(frame_w * CENTER_X_MIN) or center_x > int(frame_w *
CENTER_X_MAX):
                continue
            cv.rectangle(display, (x,y), (x+w, y+h), COLOR_BOX, 2)
            fx, fy = get_foot_point((x, y, w, h))
            cv.circle(display, (fx, fy), 5, COLOR_FOOT, -1)
            rx, ry = transform_point(M, fx, fy)
            if 0 <= rx < COURT_W and 0 <= ry < COURT_H:
                heatmap_acc[ry, rx] += 1
                positions_log.append((rx, ry, frame_count))
        elapsed_min = frame_count / (fps * 60)
        cv.putText(display,
```

```
                    f'Frame: {frame_count}/{max_frames} | '
                    f'Time: {elapsed_min:.1f}/{DURATION_MIN} min | '
                    f'Players: {len(boxes)}',
                    (10, 30), cv.FONT_HERSHEY_SIMPLEX, 0.65, (150,255,0), 2)
        h_disp, w_disp = display.shape[:2]
        scale = DISPLAY_W / w_disp
        display_small = cv.resize(display, (DISPLAY_W, int(h_disp * scale)))
        cv.imshow('Badminton Tracking', display_small)
        if cv.waitKey(1) & 0xFF == ord('q'):
            print('[INFO] Nguoi dung dung som.')
            break
        frame_count += 1
    cap.release()
    cv.destroyAllWindows()
    print(f'\n[INFO] Tong so diem vi tri da thu thap: {len(positions_log)}')
    if len(positions_log) == 0:
        print('[CANH BAO] Khong co du lieu vi tri. Kiem tra lai video va M.npy')
        return
    result = render_heatmap(heatmap_acc, COURT_IMG)
    cv.imwrite(OUTPUT_PATH, result)
    print(f'[OK] Da luu heatmap vao: {OUTPUT_PATH}')
    h_r, w_r = result.shape[:2]
    scale_r   = HEATMAP_DISP_W / w_r
    result_small = cv.resize(result, (HEATMAP_DISP_W, int(h_r * scale_r)))
    cv.imshow('Heatmap Ket qua', result_small)
    print('Nhan phim bat ky de thoat...')
    cv.waitKey(0)
    cv.destroyAllWindows()
if __name__ == '__main__':
    main()
```

2. Player detection function (bottom half player on camera side)

```
def detect_players(fg_mask, min_area=MIN_AREA):
    # Lọc nhiễu morphology
    kernel = cv.getStructuringElement(cv.MORPH_ELLIPSE, (7, 7))
    mask = cv.morphologyEx(fg_mask, cv.MORPH_OPEN,  kernel)
    mask = cv.morphologyEx(mask,    cv.MORPH_CLOSE, kernel)
    mask = cv.dilate(mask, kernel, iterations=2)
    contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL,
                                  cv.CHAIN_APPROX_SIMPLE)
    boxes = []
    for cnt in contours:
        area = cv.contourArea(cnt)
        if area < min_area:
            continue
        x, y, w, h = cv.boundingRect(cnt)
        if h < w:          # người đứng thì h > w
            continue
        if h < 30:         # quá thấp → không phải người
            continue
        boxes.append((x, y, w, h))
    return boxes
```

3. Heatmap rendering function

```
def render_heatmap(acc, court_img):
    HALF_Y = COURT_H // 2
    acc_half = acc[HALF_Y:, :]
    blur = cv.GaussianBlur(acc_half, (81, 81), 20)
    if blur.max() == 0:
        print('[WARN] Chua co du lieu heatmap!')
        court = np.ones((HALF_Y, COURT_W, 3), dtype=np.uint8)
        court[:] = (60, 140, 60)
        _draw_half_court_lines(court)
        return court
    blur = np.power(blur, 0.4)
    norm = cv.normalize(blur, None, 0, 255, cv.NORM_MINMAX)
    norm_uint8 = np.uint8(norm)
```

```
    colored = cv.applyColorMap(norm_uint8, cv.COLORMAP_JET)
    court = np.ones((HALF_Y, COURT_W, 3), dtype=np.uint8)
    court[:] = (60, 140, 60)
    result = cv.addWeighted(court, 0.3, colored, 0.7, 0)
    _draw_half_court_lines(result)
    _draw_legend(result)
    return result
```

4. Smoothtracker filtering program (for the large boundary only).

```
class SmoothTracker:
    def __init__(self):
        self.alpha     = SMOOTH_ALPHA
        self.miss_tol  = MISS_TOLERANCE
        self._smooth   = None   # (cx, cy, w, h) float
        self._miss     = 0
        self._last_vel = (0.0, 0.0)   # vận tốc cx, cy để dự đoán khi mất

    def _clamp(self, w, h):
        w = float(np.clip(w, MIN_BOX_W, MAX_BOX_W))
        h = float(np.clip(h, MIN_BOX_H, MAX_BOX_H))
        return w, h

    def update(self, roi_boxes):
        """
        roi_boxes : list (x,y,w,h) đã lọc ROI.
        Chỉ lấy box lớn nhất (người gần camera).
        Trả về (x,y,w,h) đã smooth hoặc None nếu mất quá lâu.
        """
        if roi_boxes:
            # Lấy box có diện tích lớn nhất
            best = max(roi_boxes, key=lambda b: b[2] * b[3])
            bx, by, bw, bh = best
            bw, bh = self._clamp(bw, bh)
            cx = float(bx + bw / 2)
            cy = float(by + bh / 2)

            if self._smooth is None:
                self._smooth = (cx, cy, bw, bh)
                self._last_vel = (0.0, 0.0)
            else:
                a = self.alpha
                scx, scy, sw, sh = self._smooth
                new_cx = a * cx  + (1-a) * scx
                new_cy = a * cy  + (1-a) * scy
                new_w  = a * bw  + (1-a) * sw
                new_h  = a * bh  + (1-a) * sh
                self._last_vel = (new_cx - scx, new_cy - scy)
                self._smooth = (new_cx, new_cy, new_w, new_h)

            self._miss = 0

        else:
            self._miss += 1
            if self._miss > self.miss_tol:
                self._smooth = None
                self._last_vel = (0.0, 0.0)
            elif self._smooth is not None:
                # Dự đoán vị trí bằng vận tốc cuối (coast)
                scx, scy, sw, sh = self._smooth
                vx, vy = self._last_vel
                # Giảm dần vận tốc khi không có detection
                vx *= 0.85
                vy *= 0.85
                self._last_vel = (vx, vy)
                self._smooth = (scx + vx, scy + vy, sw, sh)
```

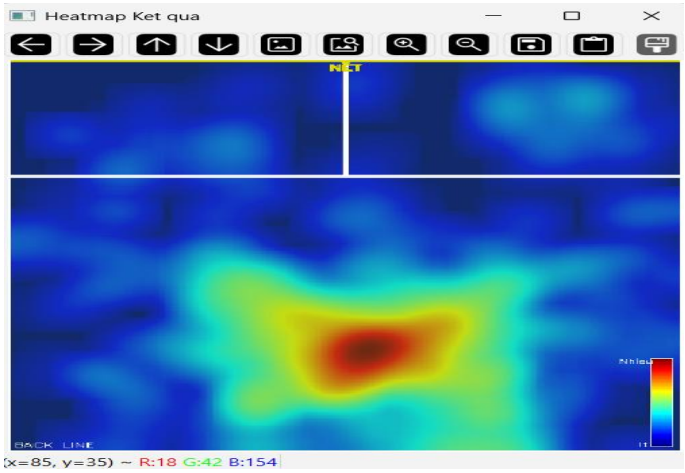
```
if self._smooth is None:
    return None

cx, cy, w, h = self._smooth
x = int(cx - w / 2)
y = int(cy - h / 2)
return (x, y, int(w), int(h))
```

VII. Result:

1. Sample tracking visualization:

Boundary box: the green rectangular box around detected player.
Foot prints: red dots marking the foot positions
Status bar: turquoise color bar on the top left of screen showing frame count, running time and player detected.

2. Heatmap result:



Heatmap after 10 minutes of running

Output file: heatmap_result.png
Court diagram: Green background with white court lines
Heatmap overlay: JET colormap showing density
Legend (on the bottom right): color bar indicate activity level (red = highest, blue = lowest)
Interpretation:

-Red zones: Player frequently moved to these areas (e.g., baseline corners, net area).

-Blue zones: Player rarely moved to these areas (e.g., sidelines, back corners).

-Other colors zones: Moderate activity areas.